UNIVERSIDAD RAFAEL NÚÑEZ

Especialización en Analítica de Datos · Tecnologías de Procesamiento de Datos

Preparación Previa — Taller 1 en GitHub Codespaces

Docente: Heyder Medrano

Cuándo: Antes de la primera sesión del Taller 1 (trabajo asincrónico, fuera del horario de clase).

Resultado esperado: Tu codespace creado, con el mes de viajes de Uber y Lyft (TLC, junio de 2025) convertido a CSV, Hadoop, Spark y Kafka funcionando, la app de viajes abierta y el verificador diciendo «LISTO para el Taller 1».

Tiempo estimado: 35 a 45 minutos. La primera vez, GitHub tarda unos 7 minutos en preparar el codespace: no cierres la pestaña mientras tanto.

Qué necesitas: Un computador con Chrome (u otro navegador actualizado), conexión estable a internet, una cuenta de GitHub. Los programas del taller se descargan en el paso 2. No instalas nada en tu equipo.

Términos: si un término no te resulta claro, búscalo en Glosario_Tecnico_Tecnologias_Procesamiento_Datos.docx (carpeta 02_Octubre/Estudiantes).

⚠ En clase no hay tiempo para preparar el codespace ni para resolver problemas de instalación. Llega con el paso 9 cumplido («RESULTADO: LISTO para el Taller 1»).

0. Antes de empezar: qué vas a hacer

En el Taller 1 vas a procesar los 19,9 millones de viajes de Uber y Lyft de junio de 2025 con cuatro herramientas: Python (pandas), Hadoop MapReduce y Spark (procesamiento por lotes), y Kafka con Spark Streaming (tiempo real). Esas herramientas necesitan un computador con bastante memoria. En vez de instalarlas en tu equipo, vas a usar un computador en la nube de GitHub, llamado codespace.

El caso y para qué sirve cada herramienta

La gerencia de operaciones necesita dos respuestas sobre los viajes de Uber y Lyft en Nueva York. Primera, con el mes completo de junio de 2025: cuántos viajes hizo cada empresa, cuánto cobró de tarifa base, cuánto pagó a sus conductores y qué parte se quedó la plataforma (Taller 1A). Segunda, mientras los viajes ocurren: en qué zonas están esperando más los pasajeros y cuánto ingresa cada empresa, minuto a minuto (Taller 1B). Cada herramienta resuelve una parte:

Herramienta	Cuándo se usa	Para qué, en el caso
Excel y Python (pandas)	Taller 1A, Parte A	Comprobar hasta dónde llega un solo computador: Excel no abre el mes (4 GB) y pandas pediría cerca de 11 GB de memoria.
Hadoop: HDFS, YARN y MapReduce	Taller 1A, Parte B	HDFS guarda el mes partido en 31 bloques; YARN reparte las tareas; MapReduce calcula por empresa viajes, tarifa, pago y parte de la plataforma sin cargar el mes en memoria. También se diseña un cálculo propio: la espera promedio por zona.
Spark	Taller 1A, Parte C	El mismo cálculo que MapReduce, en memoria y más rápido; se compara leer el CSV con leer el Parquet oficial y se mide cuánto mejora con más núcleos.
Kafka	Taller 1B, Parte A	Llevar en tiempo real los seis eventos de cada viaje (solicitud, asignación, llegada, recogida, fin y pago) entre la app del cliente, la del conductor y la central, sin perder ninguno aunque un programa se caiga.

Herramienta	Cuándo se usa	Para qué, en el caso
Spark Streaming	Taller 1B, Parte B	Leer esos eventos mientras llegan y calcular cada 10 segundos la espera por zona y los ingresos por empresa: la señal que usaría una tarifa dinámica.

En esta preparación no se hace ese análisis: se deja cada herramienta descargada, encendida y probada con una operación mínima, para que en clase todo el tiempo sea para el caso.

Términos que vas a ver

Término	Qué es	En tu codespace
Nodo	Una máquina (o un contenedor) que forma parte de un sistema distribuido.	Cada contenedor de Hadoop hace de nodo.
Clúster	Un grupo de nodos que trabajan juntos como si fueran un solo sistema.	Los cinco contenedores de Hadoop forman un clúster pequeño, todo dentro de tu codespace.
Contenedor	Un computador aislado y liviano que corre un programa con todo lo que necesita.	Hadoop, Spark, Kafka y ZooKeeper corren en contenedores.
Imagen	La plantilla de la que se crea un contenedor. Se descarga una sola vez.	apache/hadoop:3.4.3, spark:3.5.9-python3, confluentinc/cp-kafka:7.6.0 y confluentinc/cp-zookeeper:7.6.0: esta preparación las descarga para no hacerlo en clase.
Docker y docker compose	Docker crea y corre contenedores; docker compose enciende o apaga varios a la vez según el archivo docker-compose.yml de cada carpeta.	docker compose stop / down en las carpetas hadoop y kafka.
Puerto	El número por el que un programa atiende conexiones.	9870 (HDFS), 8088 (YARN), 19888 (historial), 4040 (Spark) y 8501 (app de viajes). GitHub publica cada uno en una dirección propia.

Los demás términos (bloque, NameNode, DataNode, tema, partición, offset…) se explican cuando aparecen y están en el Glosario técnico de Tecnologías de Procesamiento de Datos.

- **Qué es un codespace:** un computador Linux de 4 núcleos y 16 GB de RAM que se abre en una pestaña del navegador, con un editor (Visual Studio Code) y una terminal. Trae Docker instalado: Hadoop, Spark y Kafka corren ahí en contenedores, con las mismas imágenes y versiones que usa el taller en un computador local.
- **Qué hay en él:** tu repositorio del taller (lo creas en el paso 2) con la carpeta `tpd` (los programas del caso) y la configuración que deja todo listo: instala las librerías de Python en las versiones exactas del taller y habilita la red entre contenedores.
- **Qué vas a hacer en esta preparación:** crear tu cuenta de GitHub (paso 1), crear tu repositorio del taller (paso 2), crear tu codespace (pasos 3 y 4), descargar y convertir los datos del caso (paso 5), encender y probar Hadoop (paso 6), Spark (paso 7) y Kafka con la app de viajes (paso 8), verificar todo con un solo comando (paso 9) y apagar el codespace (paso 10).

Cuánto cuesta: nada, si cuidas la cuota gratuita

Una cuenta gratuita de GitHub incluye cada mes 120 horas-núcleo de uso y 15 GB-mes de almacenamiento. En la máquina de 4 núcleos del taller, 120 horas-núcleo equivalen a unas 30 horas de uso: alcanzan para esta preparación y

las sesiones del Taller 1. GitHub no pide tarjeta; si la cuota se agotara, simplemente detiene el codespace hasta el mes siguiente.

Regla	Por qué
Apaga el codespace al terminar cada sesión (paso 10).	Encendido consume horas aunque no lo uses. Si lo olvidas, GitHub lo apaga solo tras 30 minutos sin actividad.
No lo borres hasta terminar el Taller 1.	Ahí quedan los datos (5,5 GB) y las imágenes descargadas. Si lo borras, debes repetir esta preparación.
Usa un solo codespace.	Cada codespace guardado ocupa almacenamiento de la cuota. Para volver a entrar, abre el mismo (paso 10, «Para volver a entrar»).

1. Crear tu cuenta de GitHub

Si ya tienes una cuenta de GitHub, salta al paso 2.

1. Entra a https://github.com/signup.
2. Sigue las indicaciones: escribe tu correo, una contraseña y un nombre de usuario. También puedes registrarte con «Continue with Google».
3. Verifica tu correo: GitHub te envía un código o un enlace. Sin el correo verificado no podrás crear el codespace.
4. Cuando GitHub te pregunte por un plan, quédate con el gratuito (Free). No necesitas registrar tarjeta.

Sugerencia: anota tu nombre de usuario: aparece en la dirección de tu repositorio (github.com/tu-usuario/tpd-codespace) y en la terminal del codespace (por ejemplo, @tu-usuario).

2. Crear tu repositorio del taller

Para qué: tener en tu cuenta tu propia copia del taller: la carpeta tpd (los programas del caso) y la configuración que usa GitHub para crear el codespace con 4 núcleos y red entre contenedores. Se hace una sola vez.

Paso 2.1. Descargar y descomprimir tpd-codespace.zip

1. Entra a la página de descarga del taller: https://github.com/hmedrano-gif/tpd-materiales/releases/tag/v1 (no necesitas iniciar sesión).
2. En **Assets**, haz clic en **tpd-codespace.zip** (1). El navegador lo guarda en tu carpeta Descargas (45 KB).

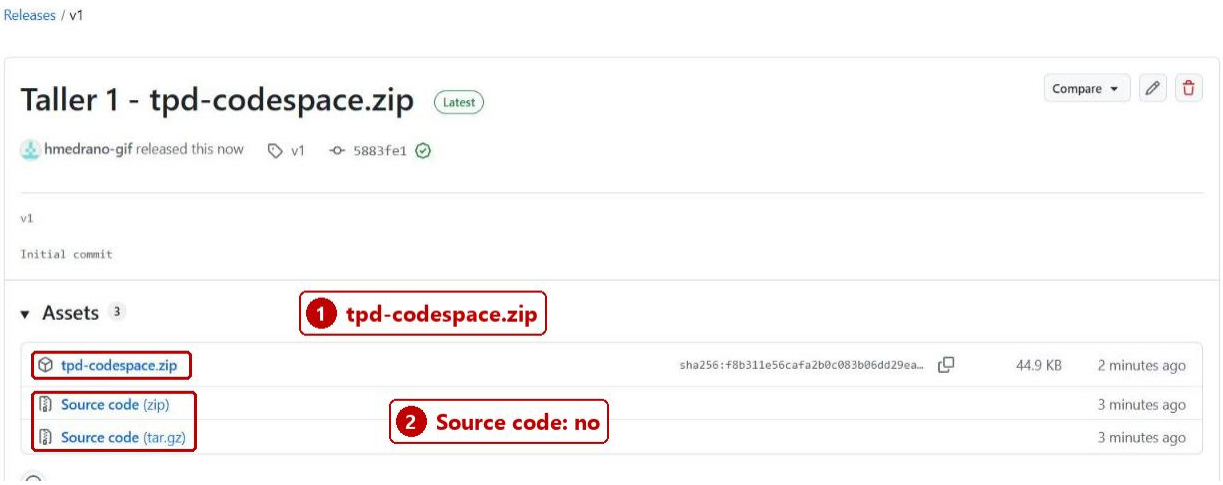


Figura 1. La página de descarga: tpd-codespace.zip (1). No uses «Source code» (2).

⚠ *Descarga tpd-codespace.zip, no «Source code (zip)»: ese otro archivo no contiene la carpeta del taller.*

3. Abre la carpeta Descargas en el Explorador de archivos.
4. Haz clic sobre el archivo (1) y pulsa **Extraer todo** en la barra de arriba (2). También funciona clic derecho → Extraer todo…

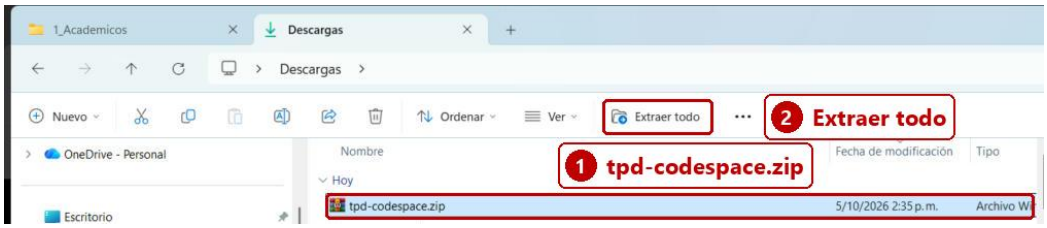


Figura 2. Selecciona tpd-codespace.zip (1) y pulsa Extraer todo (2).

5. En la ventana que aparece, deja la carpeta que propone (termina en `Downloads\tpd-codespace`) y pulsa **Extraer**.
6. Se abre la carpeta tpd-codespace con tres elementos: devcontainer.txt, README.md y la carpeta tpd (1).

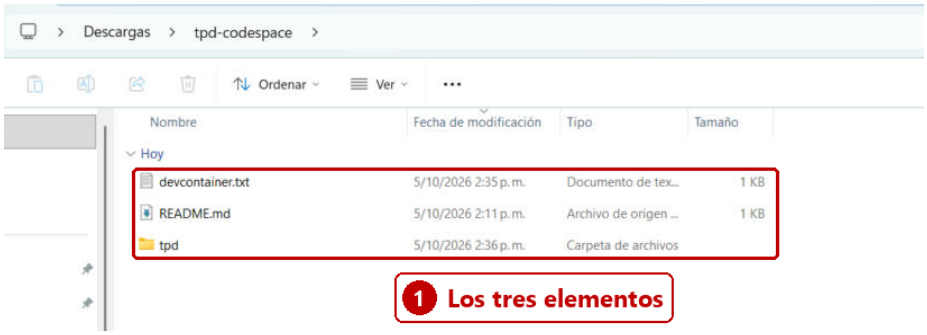


Figura 3. La carpeta descomprimida con sus tres elementos (1).

✓ **Punto de control:** ves los tres elementos. Si ves una sola carpeta llamada tpd-codespace, entra en ella: los tres elementos están adentro.

💡 **Sugerencia:** en un Mac, el doble clic sobre tpd-codespace.zip lo descomprime en la misma carpeta Descargas; en el paso 2.3 se seleccionan los tres elementos con Cmd + clic y se arrastran desde el Finder.

Paso 2.2. Crear el repositorio vacío

1. Entra a https://github.com/new (con tu cuenta de GitHub iniciada).
2. En **Repository name** escribe `tpd-codespace` (1): todo en minúsculas y con guion. Debajo aparece en verde «tpd-codespace is available».
3. En **Choose visibility** cambia Public por **Private** (2). Deja Add README en Off, No .gitignore y No license: el repositorio debe nacer vacío.
4. Pulsa **Create repository** (3).

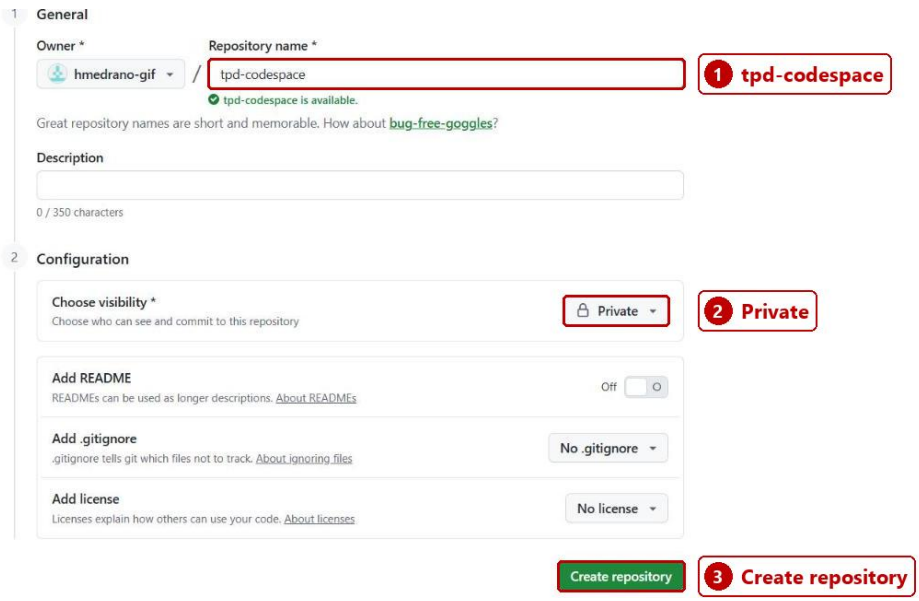


Figura 4. Nombre tpd-codespace (1), privado (2) y Create repository (3).

⚠ El nombre debe ser exactamente tpd-codespace. Todos los comandos del taller usan la ruta /workspaces/tpd-codespace: con otro nombre fallan con «No such file or directory».

Paso 2.3. Subir los archivos

1. En la página del repositorio vacío, haz clic en el enlace **uploading an existing file** (1).

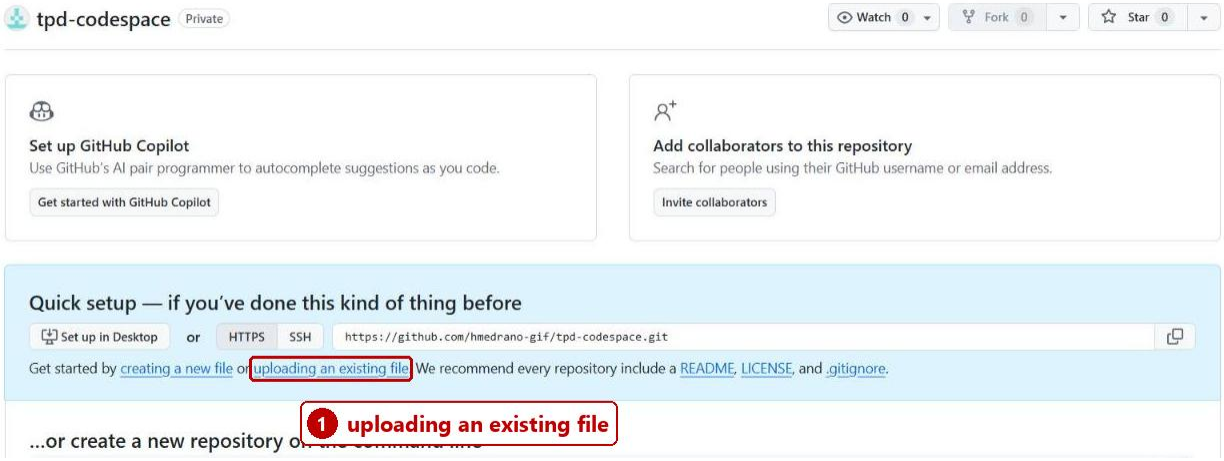


Figura 5. En el repositorio vacío, el enlace «uploading an existing file» (1).

2. Pon la ventana del Explorador (con la carpeta tpd-codespace abierta) al lado del navegador: arrastra cada ventana hacia un borde de la pantalla, o usa la tecla Windows + flecha izquierda o derecha. Selecciona los tres elementos: clic en devcontainer.txt y Mayús + clic en tpd.
3. Arrástralos al recuadro **Drag files here to add them to your repository** y suéltalos. GitHub muestra la lista: 28 archivos, con devcontainer.txt, README.md y los de tpd/ (1).

⚠ Arrastra los tres elementos de adentro, no la carpeta tpd-codespace. Si arrastras la carpeta, todo queda dentro de tpd-codespace/ y los comandos del taller no encuentran los programas.

4. Baja hasta el final de la página y pulsa **Commit changes** (2). Aparece «Processing your files…» unos segundos (en la prueba, 10 s).

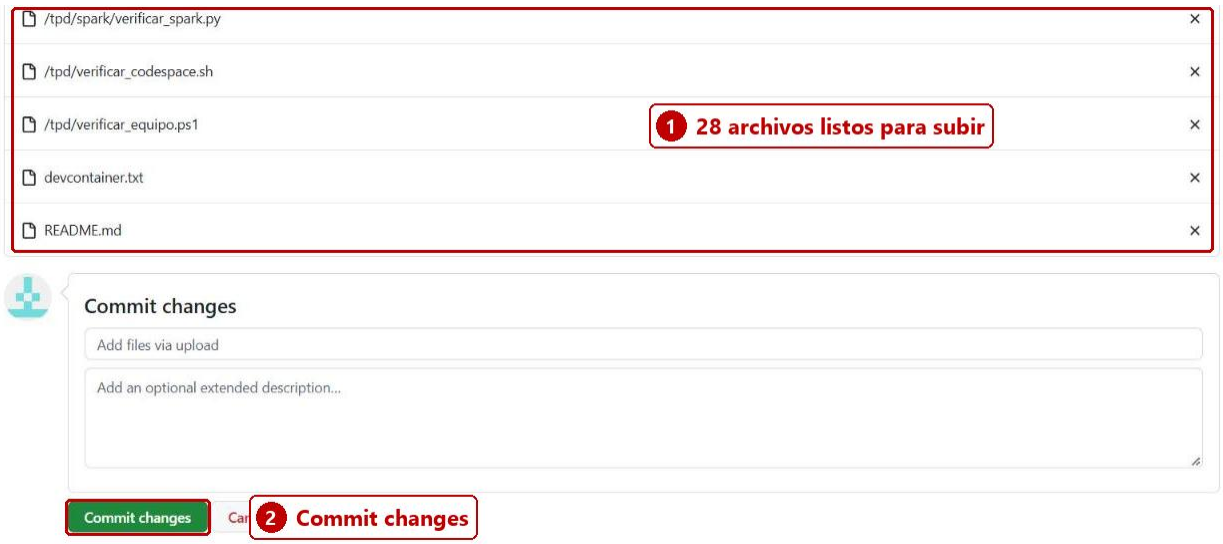


Figura 6. Los 28 archivos listos para subir (1) y el botón Commit changes (2).

✓ **Punto de control:** la portada del repositorio muestra tpd, README.md y devcontainer.txt.

Paso 2.4. Crear la configuración del codespace

GitHub no sube por arrastre las carpetas cuyo nombre empieza con punto, y la configuración del codespace debe estar en .devcontainer/devcontainer.json. Por eso viene como devcontainer.txt: vas a copiar su contenido y crear ese archivo directamente en GitHub.

1. En tu repositorio, haz clic en **devcontainer.txt** (1). Arriba a la derecha del contenido, pulsa el botón **Copy raw file** (2): el contenido queda copiado.



Figura 7. Abre devcontainer.txt (1) y cópialo con Copy raw file (2).

2. Vuelve a la portada del repositorio (clic en **tpd-codespace**, arriba). Pulsa **Add file** (1) → **Create new file** (2).

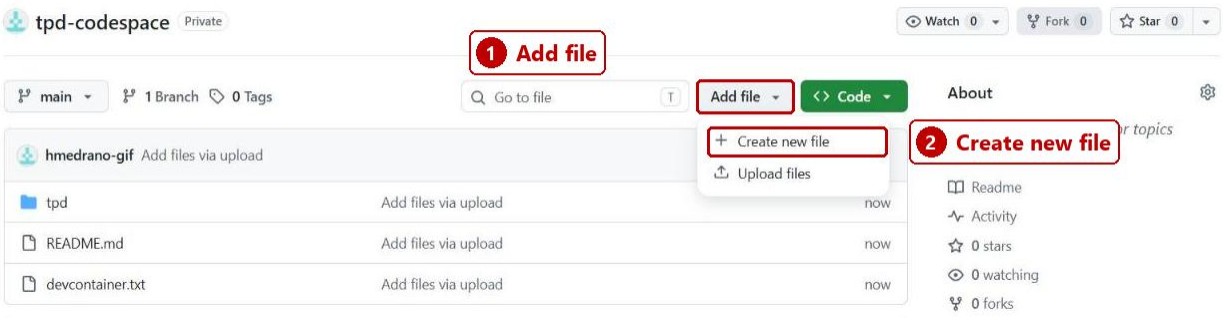


Figura 8. Add file (1) → Create new file (2).

3. En **Name your file…** escribe `.devcontainer/devcontainer.json` (1). Al escribir la barra /, GitHub convierte .devcontainer en carpeta.
4. Haz clic en el área grande de contenido y pega con `Ctrl+V` (2): aparecen 8 líneas; la primera es { y la última }.
5. Pulsa **Commit changes…** (3), arriba a la derecha.



Figura 9. Nombre .devcontainer/devcontainer.json (1), contenido pegado (2) y Commit changes… (3).

6. En la ventana de confirmación deja todo como está («Commit directly to the main branch») y pulsa **Commit changes** (1).

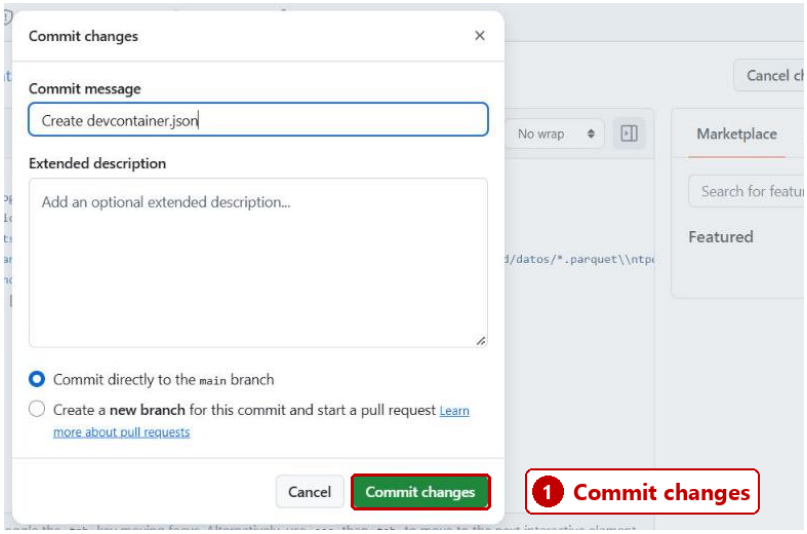


Figura 10. Confirma con Commit changes (1).

7. Vuelve a la portada del repositorio. Ahora tiene cuatro elementos, con la carpeta .devcontainer arriba (1), y junto al nombre dice Private (2).

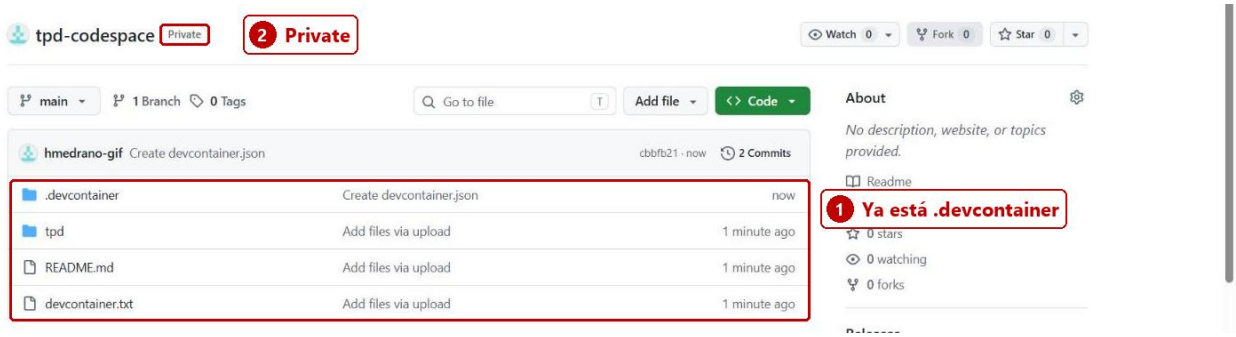


Figura 11. Tu repositorio del taller, listo para crear el codespace: aparece .devcontainer (1) y es privado (2).

✓ **Punto de control:** la portada muestra .devcontainer, tpd, README.md y devcontainer.txt. Si no aparece .devcontainer, repite el paso 2.4: sin ese archivo el codespace se crea con 2 núcleos y Hadoop y Kafka no funcionan.

3. Crear tu codespace

Para qué: crear tu computador en la nube a partir de tu repositorio del taller. Se hace una sola vez.

1. Abre tu repositorio: `https://github.com/<tu usuario>/tpd-codespace` (es la página donde terminaste el paso 2).
2. Pulsa el botón verde **Code** (1), luego la pestaña **Codespaces** (2) y el botón **Create codespace on main** (3).

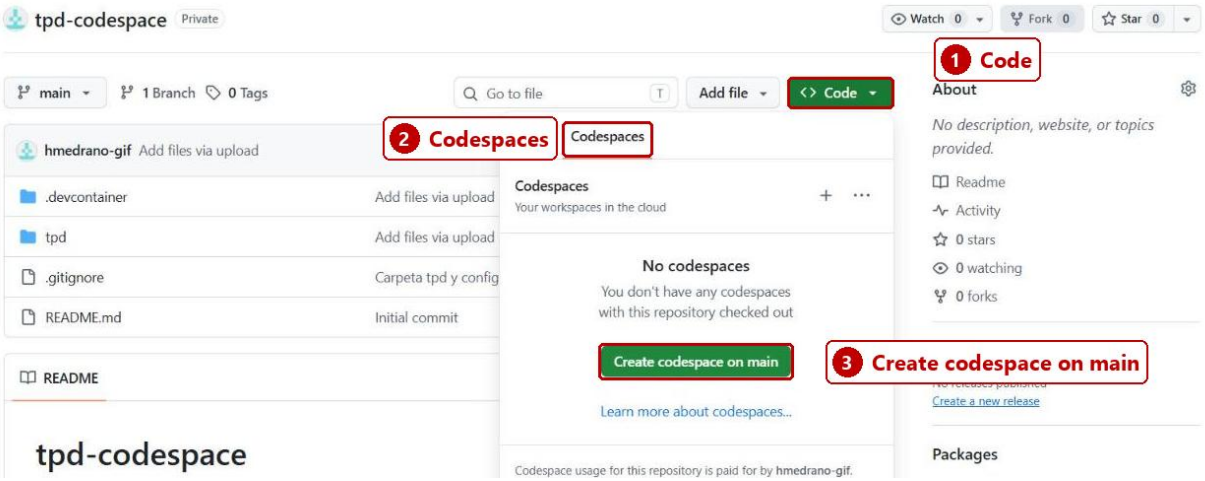


Figura 12. En el repositorio: Code (1) → Codespaces (2) → Create codespace on main (3).

⚠ Pulsa «Create codespace on main» una sola vez. Si ya tienes un codespace de este repositorio, aparece en esa misma lista: ábrelo en vez de crear otro (cada uno ocupa cuota).

3. Se abre una pestaña nueva con el editor y abajo a la derecha el aviso «Setting up remote connection: Building codespace…». Espera sin cerrar la pestaña: la primera vez tarda unos 7 minutos (en la prueba: 7 minutos y 20 segundos).

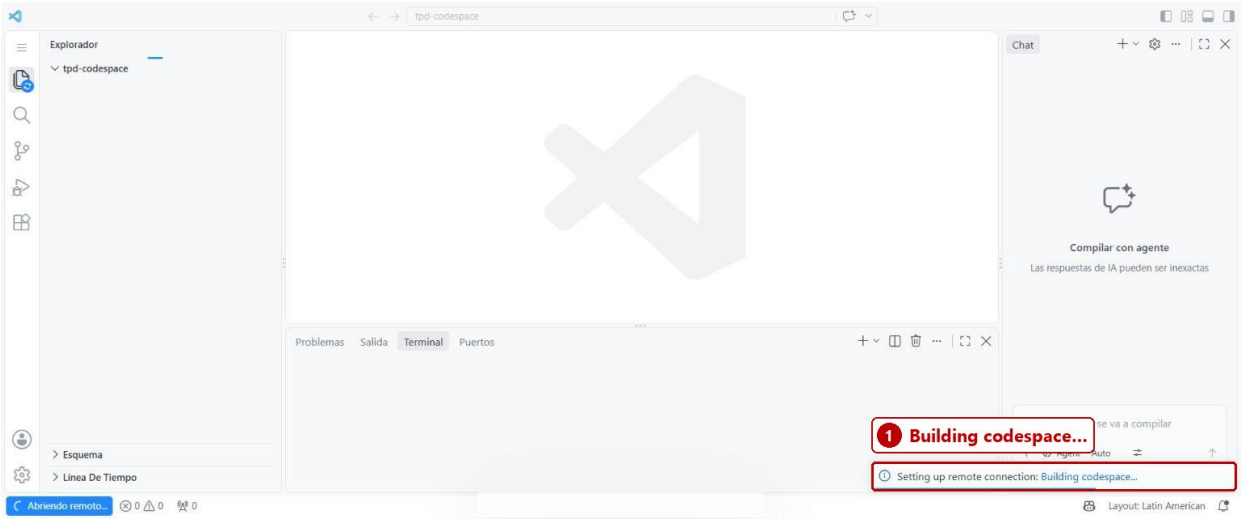


Figura 13. Primer arranque: GitHub prepara el codespace (1). Puede tardar unos 7 minutos.

4. La pregunta de confianza y la terminal

1. Durante el arranque aparece la pregunta «¿Confía en los autores de los archivos de esta carpeta?». Pulsa **Confiar en la carpeta y continuar** (1): los archivos son los del taller.



Figura 14. Pulsa «Confiar en la carpeta y continuar» (1).

2. Espera a que la terminal de abajo muestre **Finished configuring codespace.** (1). En ese momento ya están instaladas las librerías de Python y habilitada la red entre contenedores.
3. Abre una terminal nueva con el botón **+** del panel de terminales (2). Todos los comandos de esta guía se escriben en esa terminal.

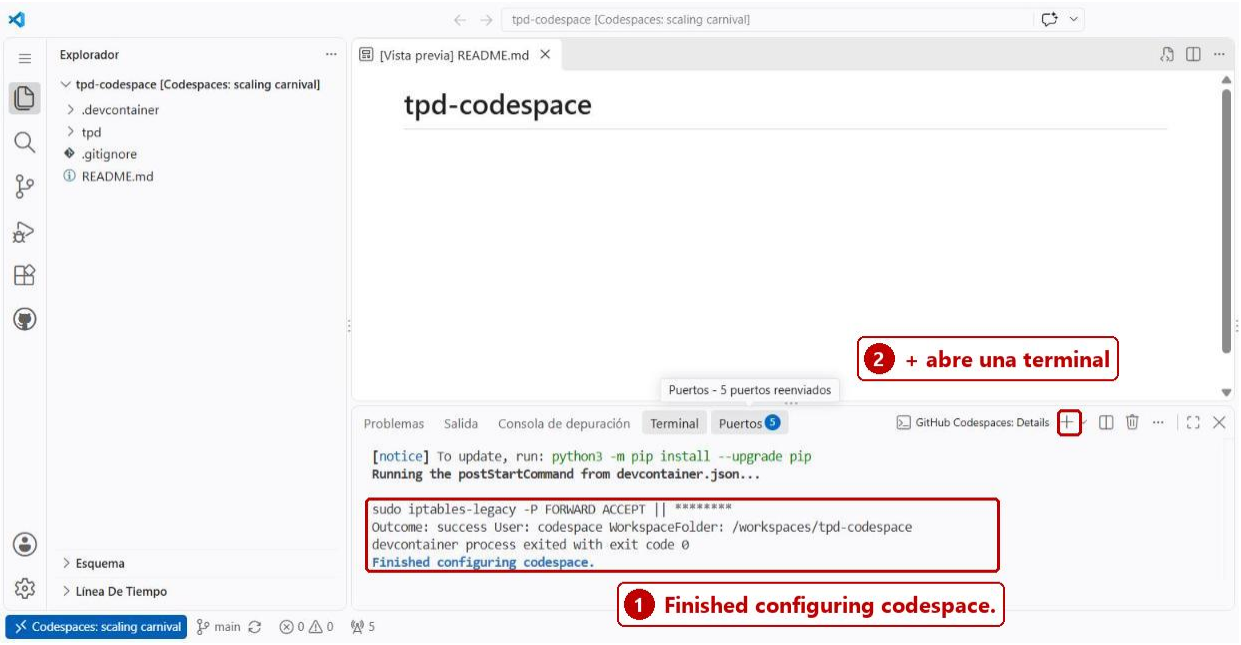


Figura 15. El codespace está listo (1). Con + (2) abres la terminal donde vas a trabajar.

Si al abrir el codespace no ves la terminal de la creación sino directamente una terminal con el mensaje «Welcome to Codespaces!» y el indicador `@<tu usuario> → /workspaces/tpd-codespace (main) $`, la configuración ya terminó mientras la pestaña cargaba: trabaja en esa terminal y sigue con el paso 5. El verificador del paso 9 confirma que quedó bien (núcleos, librerías y red entre contenedores).

Si algo de la configuración falló, el verificador lo marca con [FALTA]. Para ver qué pasó durante la creación: `F1` → escribe `view creation log` → «Codespaces: View Creation Log», y envía al docente una captura del final de ese registro.

La terminal muestra un indicador como este (con tu usuario):

```
@<tu usuario> → /workspaces/tpd-codespace (main) $
```

- Copia cada comando del recuadro negro y pégalo en la terminal con `Ctrl+V` (o clic derecho → Pegar).
- Presiona Enter y espera a que vuelva a aparecer el indicador antes de pegar el siguiente comando.
- **Si no ves el indicador, la terminal está ocupada:** hay un programa corriendo y lo que escribas le llega a ese programa, no se ejecuta como comando. Algunos programas no terminan solos porque se quedan esperando, como la app de viajes. Para detenerlos y recuperar la terminal se presiona `Ctrl+C`. Si necesitas el programa abierto y también escribir comandos, abre otra terminal con +.

⚠ *No cierres la pestaña del codespace mientras un comando está corriendo: el comando se detiene.*

5. Descargar y convertir los datos del caso

Para qué: tener el mes completo de viajes (19.868.009) en el formato que usan Hadoop y Excel. La Comisión de Taxis y Limusinas de Nueva York (TLC) publica el mes en formato Parquet (491 MB); el script lo descarga y lo convierte a CSV (4,04 GB).

Paso 5.1. El mes completo

Qué hace cada comando:

- `cd tpd/datos`: entra a la carpeta de los datos. El indicador pasa a terminar en /tpd/datos (main) $.
- `python preparar_datos.py`: ejecuta el script que prepara los datos del caso. Hace tres cosas, en orden:
  - **Descarga el Parquet oficial:** baja fhvhv_tripdata_2025-06.parquet (491 MB) del servidor de la TLC y comprueba que llegó completo (tamaño exacto en bytes). Si ya estaba descargado, no lo vuelve a bajar.
  - **Lo convierte a CSV por partes:** el Parquet viene en 19 grupos de filas; el script lee un grupo a la vez y lo agrega a fhvhv_2025-06.csv. Así la memoria nunca pasa de unos cientos de MB, cuando cargar el mes entero de una vez pediría cerca de 12 GB. El CSV conserva las 25 columnas en su orden oficial, del que dependen los programas de Hadoop y Spark del taller.
  - **Comprueba el resultado:** cuenta los viajes escritos y los compara con el total oficial (19.868.009). Si coinciden, imprime «Punto de control correcto»; si no, avisa que hay que borrar el CSV y repetir.

Salida esperada *(prueba del 5 de octubre; los segundos cambian un poco en cada corrida)*

```
Descargando https://d37ci6vzurychx.cloudfront.net/trip-data/fhvhv_tripdata_2025-06.parquet (491 MB)...
       0 de 491 MB
      49 de 491 MB
…
     491 de 491 MB
Descargado en 4 s
Parquet: 19,868,009 viajes, 25 columnas, 19 grupos de filas
  grupo  1 de 19: 1,048,576 viajes escritos
…
  grupo 19 de 19: 19,868,009 viajes escritos
fhvhv_2025-06.csv: 19,868,009 viajes, 4.04 GB, 70 s
Punto de control correcto: la cantidad de viajes es la esperada para esta porción.
```

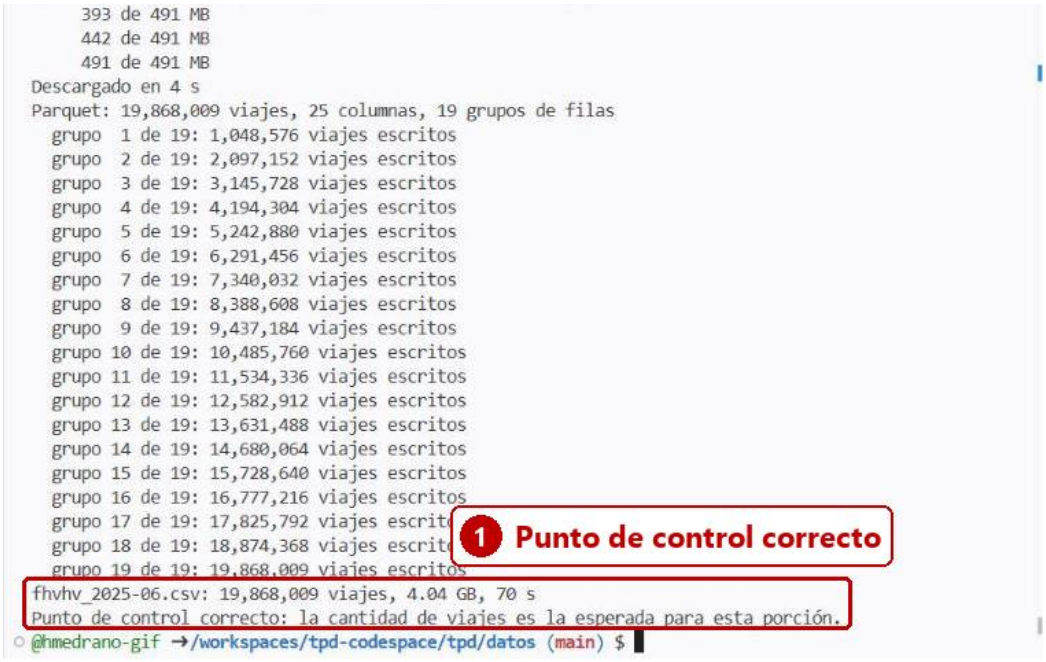


Figura 16. El mes quedó convertido: 19.868.009 viajes y «Punto de control correcto» (1).

✓ **Punto de control:** la última línea dice «Punto de control correcto». Después del cd, el indicador termina en /tpd/datos (main) $.

Para qué sirve cada archivo en el Taller 1:

Archivo	Tamaño	Dónde se usa
fhvhv_tripdata_2025-06.parquet	0,49 GB	Spark lo lee directamente en el Taller 1A (para comparar con el CSV), y el simulador de viajes lo usa en el Taller 1B.
fhvhv_2025-06.csv	4,04 GB	Es el archivo que se carga en HDFS y procesa Hadoop MapReduce en el Taller 1A; Spark también lo lee para comparar tiempos.
fhvhv_2025-06_semana.csv	0,93 GB	Lo genera el paso 5.2: es la porción con la que el docente muestra los límites de Excel.

Paso 5.2. La semana (para el paso de Excel del taller)

Excel no abre archivos de más de 2 GB, así que el mes completo (4,04 GB) ni siquiera entra. En el taller, el docente muestra ese límite con una porción más pequeña: los viajes del 2 al 8 de junio.

- `python preparar_datos.py --semana`: el mismo script, con la opción --semana. Como el Parquet ya está descargado, no vuelve a bajarlo: lo recorre por grupos, se queda solo con los viajes recogidos entre el 2 y el 8 de junio, y los escribe en fhvhv_2025-06_semana.csv con las mismas 25 columnas. Al final compara con el total esperado de la semana (4.576.158 viajes).

Genérala ahora:

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/datos · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/datos (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/datos (main) $` (Ctrl+V o clic derecho) y presiona Enter.

```
python preparar_datos.py --semana
```

Salida esperada *(solo las últimas líneas; reutiliza el Parquet ya descargado)*

```
…
  grupo 19 de 19: 4,576,158 viajes escritos
fhvhv_2025-06_semana.csv: 4,576,158 viajes, 0.93 GB, 13 s
Punto de control correcto: la cantidad de viajes es la esperada para esta porción.
```

✓ **Punto de control:** 4.576.158 viajes y «Punto de control correcto».

6. Encender y probar Hadoop

En el Taller 1: en la Parte B del Taller 1A cargas en HDFS el mes completo (fhvhv_2025-06.csv, 4,04 GB), HDFS lo parte en 31 bloques y MapReduce calcula por empresa los viajes, la tarifa base, el pago a los conductores y la parte de la plataforma. Aquí solo compruebas que el clúster arranca y que HDFS guarda un archivo pequeño: zonas.csv, la tabla de las 265 zonas de Nueva York que también usa el taller.

Para qué: descargar la imagen de Hadoop (unos 750 MB) ahora y no en clase, y comprobar que el clúster arranca completo.

Paso 6.1. Encender Hadoop con el script del taller

`encender_hadoop.sh` hace lo mismo que `docker compose up -d` (enciende los cinco contenedores: namenode, datanode, resourcemanager, nodemanager e historyserver), pero no termina hasta que el clúster está listo de verdad. Revisa cuatro condiciones:

- **NameNode responde:** el servicio que guarda el índice de HDFS ya acepta conexiones.

- **DataNode registrado:** el nodo que guarda los bloques ya se reportó. Sin él, cargar un archivo falla con «0 datanode(s) running».
- **HDFS fuera del modo seguro:** al arrancar, HDFS pasa unos segundos en solo lectura (safe mode). Un job lanzado en ese momento falla con «Cannot delete /tmp/hadoop-yarn/staging…».
- **YARN con su nodo activo:** YARN es el administrador de recursos de Hadoop: decide cuánta memoria y cuántos núcleos recibe cada tarea. Su ResourceManager reparte el trabajo y su NodeManager lo ejecuta. En el Taller 1A, YARN reparte las 31 tareas map del mes según la memoria disponible (con 4 GB caben 2 a la vez). Esta condición confirma que el ResourceManager ya ve al NodeManager; sin él, un job se queda en «map 0% reduce 0%».

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/datos · el indicador se ve así: `@<tu usuario> → /workspaces/tpd-codespace/tpd/datos (main) $`

▶ **Qué hacer:** son 2 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `@<tu usuario> → /workspaces/tpd-codespace/tpd/datos (main) $` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez.

Comando 1 de 2

```
cd ../hadoop
```

Comando 2 de 2

```
bash encender_hadoop.sh
```

Salida esperada *(primera vez, con la descarga de la imagen; las siguientes veces tarda menos)*

```
Encendiendo Hadoop (namenode, datanode, resourcemanager, nodemanager, historyserver)...
 namenode Pulled
…
 Container hadoop-namenode-1  Started
 …
  NameNode responde                                    OK (67 s)
  DataNode registrado (Live datanodes 1)               OK (69 s)
  HDFS fuera del modo seguro (Safe mode is OFF)        OK (71 s)
  YARN con su nodo activo                              OK (72 s)

HADOOP LISTO en 72 s · consolas: HDFS puerto 9870 · YARN puerto 8088 (pestaña Puertos)
```



Figura 17. Las cuatro condiciones en OK y «HADOOP LISTO» (1).

✓ **Punto de control:** las cuatro líneas dicen OK y la última empieza con «HADOOP LISTO».

⚠ En el taller enciende Hadoop siempre con bash encender_hadoop.sh, no con docker compose up -d: así nunca lanzarás un job mientras HDFS está en modo seguro.

Paso 6.2. Probar HDFS y el Python del nodo

HDFS (Hadoop Distributed File System) es el disco de Hadoop: guarda cada archivo cortado en bloques de 128 MB repartidos entre los DataNodes, y el NameNode lleva el índice de qué bloque está dónde. Es un sistema de archivos aparte del codespace: un archivo que está en tu carpeta tpd no está en HDFS hasta que lo copias con put. En el Taller 1A copiarás así el mes completo (4,04 GB, 31 bloques); aquí lo pruebas con zonas.csv (12 KB).

El Python del nodo: en el Taller 1A el cálculo por empresa se escribe en Python (mapper.py y reducer.py), pero no corre en el codespace: lo ejecuta el NodeManager, el contenedor de YARN que hace las tareas. Por eso hay que comprobar que ese contenedor tiene su propio Python.

Qué hace cada comando (todos empiezan con docker compose exec -T <contenedor>, que ejecuta el comando dentro de ese contenedor):

- `hdfs dfs -mkdir -p /prueba`: crea la carpeta /prueba en HDFS (no en el codespace).
- `hdfs dfs -put -f /opt/codigo/zonas.csv /prueba/`: copia zonas.csv a HDFS. /opt/codigo es como el contenedor ve tu carpeta tpd/hadoop; -f reemplaza el archivo si ya estaba.
- `hdfs dfs -ls /prueba`: lista la carpeta de HDFS para confirmar que el archivo llegó, con su tamaño en bytes.
- `python3 --version` (en el contenedor nodemanager): muestra la versión de Python con la que se ejecutarán mapper.py y reducer.py.

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/hadoop · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/hadoop (main) $`

▶ **Qué hacer:** son 4 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/hadoop (main) $` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez.

Comando 1 de 4

```
docker compose exec -T namenode hdfs dfs -mkdir -p /prueba
```

Comando 2 de 4

```
docker compose exec -T namenode hdfs dfs -put -f /opt/codigo/zonas.csv /prueba/
```

Comando 3 de 4

```
docker compose exec -T namenode hdfs dfs -ls /prueba
```

Comando 4 de 4

```
docker compose exec -T nodemanager python3 --version
```

Salida esperada *(la fecha y la hora serán las de tu prueba)*

```
Found 1 items
-rw-r--r--   1 hadoop supergroup      12331 2026-10-05 18:21 /prueba/zonas.csv
Python 3.10.12
```

✓ **Punto de control:** aparece /prueba/zonas.csv con 12331 bytes y «Python 3.x».

Paso 6.3. Apagar Hadoop

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/hadoop · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/hadoop (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/hadoop (main) $` (Ctrl+V o clic derecho) y presiona Enter.

```
docker compose stop
```

Salida esperada *(los cinco contenedores terminan en Stopped)*

```
 Container hadoop-…-1  Stopping
 …
Container hadoop-namenode-1  Stopped
Container hadoop-resourcemanager-1  Stopped
```

Sugerencia: docker compose stop apaga sin borrar: lo que cargaste en HDFS sigue ahí la próxima vez.

7. Probar Spark

En el Taller 1: en la Parte C del Taller 1A, Spark repite el cálculo por empresa de Hadoop leyendo el CSV del mes y luego el Parquet oficial, y se comparan tiempos y datos leídos. Aquí solo compruebas que Spark arranca dentro de su contenedor y cuenta un millón de filas.

Para qué: descargar la imagen de Spark ahora y comprobar que Spark arranca y procesa datos.

Qué hace cada comando:

- `cd ../spark`: pasa de la carpeta hadoop a la carpeta spark.
- `docker run --rm … spark:3.5.9-python3`: crea un contenedor de Spark a partir de su imagen (la descarga la primera vez) y lo borra al terminar (--rm).
- `-v /workspaces/tpd-codespace/tpd:/opt/tpd -w /opt/tpd/spark`: el contenedor ve tu carpeta tpd como /opt/tpd y trabaja dentro de su carpeta spark; así lee los programas y los datos del caso sin copiarlos.
- `spark-submit verificar_spark.py`: ejecuta con Spark un programa mínimo que arranca una sesión de Spark, cuenta un rango de un millón de números e imprime la versión. En el Taller 1A, el mismo spark-submit ejecutará job_spark.py sobre el mes completo.

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/hadoop · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/hadoop (main) $`

▶ **Qué hacer:** son 2 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/hadoop (main) $` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

Comando 1 de 2

```
cd ../spark
```

Comando 2 de 2

```
docker run --rm -v /workspaces/tpd-codespace/tpd:/opt/tpd -w /opt/tpd/spark \
    spark:3.5.9-python3 /opt/spark/bin/spark-submit verificar_spark.py
```

Salida esperada *(la primera vez descarga la imagen; Spark escribe muchas líneas INFO: la que importa es la última)*

```
Unable to find image 'spark:3.5.9-python3' locally
3.5.9-python3: Pulling from library/spark
…
Status: Downloaded newer image for spark:3.5.9-python3
… (líneas INFO y WARN de Spark)
Spark 3.5.9 funcionando: contó 1,000,000 filas.
```

✓ **Punto de control:** la última línea dice «Spark 3.5.9 funcionando: contó 1,000,000 filas.». Un error real diría ERROR, Exception o Traceback.

8. Kafka y la app de viajes

En el Taller 1: en el Taller 1B pides un viaje en la app (como cliente y como conductor): cada paso del viaje se publica como un evento en el tema «viajes» de Kafka; la central lo lee al instante, un consumidor mide cuántos eventos por segundo pasan, y Spark Streaming calcula la espera por zona mientras llegan. Aquí compruebas por separado esas tres piezas: que Python publica en Kafka, que Spark lee de Kafka y que la app abre y su evento llega a la central.

Para qué: descargar las imágenes de Kafka y ZooKeeper, comprobar que Python publica eventos, que Spark los lee y que la app de viajes abre en tu navegador.

Paso 8.1. Encender Kafka con el script del taller

`encender_kafka.sh` enciende Kafka, ZooKeeper y un contenedor de Spark, y espera a que Kafka acepte conexiones. Si al reabrir el codespace Kafka queda reiniciándose (ZooKeeper conserva el registro de la sesión anterior y los programas fallan con «NoBrokersAvailable»), el script lo detecta en 60 segundos, recrea los contenedores y vuelve a esperar.

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/spark · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/spark (main) $`

▶ **Qué hacer:** son 2 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/spark (main) $` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez.

Comando 1 de 2

```
cd ../kafka
```

Comando 2 de 2

```
bash encender_kafka.sh
```

Salida esperada *(la primera vez «temas actuales» está vacío; en el taller aparecerá el tema viajes)*

```
Encendiendo Kafka, ZooKeeper y Spark...
…
 Container kafka  Started
  Kafka acepta conexiones (localhost:9092)                 OK (41 s)

KAFKA LISTO en 41 s · temas actuales:
```

✓ **Punto de control:** la línea de Kafka dice OK y aparece «KAFKA LISTO».

⚠ *En el taller, enciende Kafka siempre con bash encender_kafka.sh antes de abrir la app, el consumidor o el simulador.*

Paso 8.2. Probar que Python publica y que Spark lee

Qué hace cada comando:

- `python -c "from kafka import KafkaProducer; …"`: un productor de Kafka escrito en Python, en una sola línea. Se conecta al broker (localhost:9092), publica el mensaje «hola desde Codespaces» en el tema «verificacion» y escribe «Productor OK». Así publica la app de viajes cada evento del viaje.
- `docker compose exec -T spark … spark-submit … verificar_conector.py`: dentro del contenedor spark, ejecuta un programa que lee con Spark todo el tema «verificacion» y cuenta los mensajes. --packages descarga el conector Spark-Kafka, la pieza que le permite a Spark leer de Kafka (la primera vez tarda; luego queda guardado). Spark Streaming usará ese mismo conector en el Taller 1B.

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/kafka · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $`

▶ **Qué hacer:** son 2 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

Comando 1 de 2

```
python -c \
    "from kafka import KafkaProducer; p=KafkaProducer(bootstrap_servers='localhost:9092');
p.send('verificacion', b'hola desde Codespaces'); p.flush(); print('Productor OK')"
```

Comando 2 de 2

```
docker compose exec -T spark /opt/spark/bin/spark-submit --conf \
    spark.jars.ivy=/tmp/.ivy2 --packages \
    org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.9 \
    /opt/kafka-taller/verificar_conector.py
```

Salida esperada *(el segundo comando descarga el conector Spark-Kafka la primera vez)*

```
Productor OK
Ivy Default Cache set to: /tmp/.ivy2/cache
…
org.apache.spark#spark-sql-kafka-0-10_2.12 added as a dependency
… (descarga del conector y líneas INFO)
Conector Spark-Kafka OK: 1 mensaje(s) en el topic 'verificacion': ['hola desde Codespaces']
```

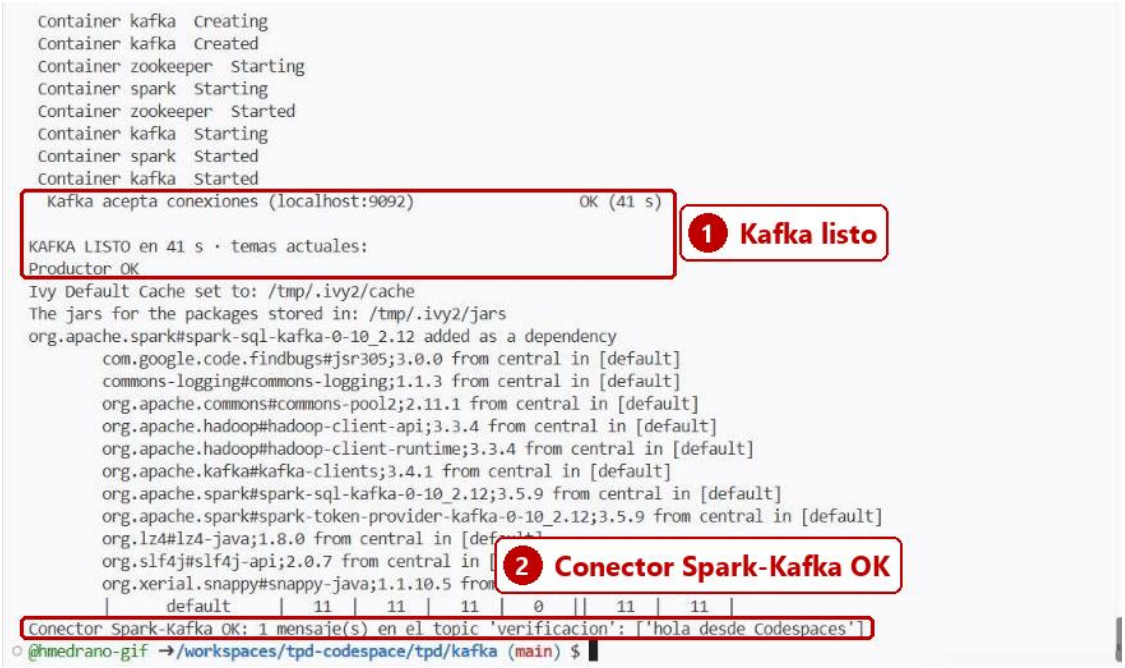


Figura 18. Kafka listo y «Productor OK» (1); Spark lee el mensaje de Kafka (2).

✓ **Punto de control:** aparecen «Productor OK» y al final «Conector Spark-Kafka OK».

Paso 8.3. Abrir la app de viajes

`streamlit run app_viajes.py --server.headless true`: arranca la app de viajes (escrita en Python con Streamlit) como un servidor web en el puerto 8501. --server.headless true evita que intente abrir un navegador dentro del codespace.

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/kafka · el indicador se ve así: `@<tu usuario> → /workspaces/tpd-codespace/tpd/kafka (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> → /workspaces/tpd-codespace/tpd/kafka (main) $` (Ctrl+V o clic derecho) y presiona Enter.

```
streamlit run app_viajes.py --server.headless true
```

Salida esperada *(las direcciones IP serán otras; no las uses: abre la app con el botón de GitHub)*

```
Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://10.0.12.17:8501
  External URL: http://135.237.130.228:8501
```

⚠ Mientras la app corre, esta terminal queda ocupada (no vuelve el indicador): lo que escribas ahí no se ejecuta. No escribas el comando del paso 8.4 hasta haber presionado Ctrl+C.

Abajo a la derecha aparece el aviso «La aplicación que se ejecuta en el puerto 8501 está disponible». Pulsa **Abrir en el navegador** (1).



Figura 19. GitHub publica la app en una dirección propia: pulsa «Abrir en el navegador» (1).

⚠ No escribas localhost:8501 ni las direcciones IP que muestra la terminal: desde tu navegador no funcionan. Si el aviso ya se cerró, abre una pestaña nueva con la dirección de tu codespace cambiando el final «.github.dev» por «-8501.app.github.dev» (ejemplo: scaling-carnival-4q7w667p4vgph5qwv.github.dev → scaling-carnival-4q7w667p4vgph5qwv-8501.app.github.dev).

La app muestra las tres miradas de un viaje: el cliente que lo pide, el conductor que lo acepta y la central de operaciones que ve todo. No se comunican directamente: cada acción se publica como un evento en Kafka y cada pantalla se entera leyéndolo. Abrir la app no prueba nada todavía; la prueba es pedir un viaje:

1. En la app, deja los datos de ejemplo (María, Uber, de Midtown Center a LaGuardia Airport) y pulsa el botón rojo **Pedir viaje** (abajo, en la columna Cliente). No marques «Trabajo sin pareja».
2. Sube en la página: el estado del cliente pasa a «solicitado» (2) y la Central de operaciones cuenta 1 evento recibido y 1 viaje solicitado (1). El evento viajó por Kafka.



Figura 20. La central recibió el evento por Kafka (1); el viaje quedó «solicitado» (2).

✓ **Punto de control:** la Central de operaciones muestra «Eventos recibidos de Kafka» 1 y «Viajes solicitados» 1. Ese 1 prueba la cadena completa: la app publicó el evento, Kafka lo guardó en el tema «viajes» y la central lo leyó desde ahí.

Sugerencia: no hace falta pulsar «Aceptar» en la columna Conductor. Si lo pulsas, verás un segundo evento («asignado») y el estado del cliente cambia: es lo que harás completo en el Taller 1B.

Paso 8.4. Cerrar la app y apagar Kafka

1. Cierra la pestaña de la app. Vuelve a la terminal, haz clic dentro y presiona `Ctrl+C`: aparece «Stopping...» y vuelve el indicador …/tpd/kafka (main) $. Solo entonces pega el comando siguiente.
2. Apaga Kafka con down (no con stop): así la próxima vez arranca limpio, sin el registro viejo en ZooKeeper.

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/kafka · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $` (Ctrl+V o clic derecho) y presiona Enter.

```
docker compose down
```

Salida esperada *(down borra los contenedores de Kafka, no las imágenes descargadas)*

```
 Container kafka  Removed
 …
Container spark  Removed
Network kafka_default  Removing
Network kafka_default  Removed
```

9. Verificar todo con un solo comando

Para qué: `verificar_codespace.sh` revisa en segundos todo lo que preparaste: la máquina (núcleos, RAM, disco), Docker y las cuatro imágenes, la red entre contenedores, Python y sus librerías, los 21 archivos de la práctica y los tres archivos de datos con su tamaño exacto. Solo mira: no instala ni enciende nada.

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/kafka · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $` (Ctrl+V o clic derecho) y presiona Enter.

```
bash /workspaces/tpd-codespace/tpd/verificar_codespace.sh
```

Salida esperada *(prueba del 5 de octubre; el disco libre puede variar)*

```
Verificación del codespace para el Taller 1 · Tecnologías de Procesamiento de Datos
Carpeta de trabajo revisada: /workspaces/tpd-codespace/tpd

1. Máquina
  [OK]    4 núcleos
  [OK]    RAM: 15.6 GB
  [OK]    Disco libre: 11 GB

2. Docker
  [OK]    Docker encendido (motor 28.1.1-1)
  [OK]    Imagen apache/hadoop:3.4.3
  [OK]    Imagen spark:3.5.9-python3
  [OK]    Imagen confluentinc/cp-kafka:7.6.0
  [OK]    Imagen confluentinc/cp-zookeeper:7.6.0
  [OK]    Red entre contenedores habilitada

3. Python y librerías
  [OK]    Python 3.12.1
  [OK]    pyarrow 24.0.0
  [OK]    pandas 2.3.3
  [OK]    streamlit 1.56.0
  [OK]    kafka-python 2.3.2

4. Carpeta de trabajo /workspaces/tpd-codespace/tpd
  [OK]    Los 21 archivos de la práctica están en su lugar

5. Datos del caso
  [OK]    fhvhv_tripdata_2025-06.parquet (0.49 GB, tamaño exacto)
  [OK]    fhvhv_2025-06_semana.csv (0.93 GB, tamaño exacto)
  [OK]    fhvhv_2025-06.csv (4.04 GB, tamaño exacto)

RESULTADO: LISTO para el Taller 1 (0 avisos).
```



Figura 21. La verificación termina con «RESULTADO: LISTO para el Taller 1» (1).

✓ **Punto de control:** la última línea dice «RESULTADO: LISTO para el Taller 1». Si alguna línea dice [FALTA], ahí mismo se indica qué hacer; corrígelo y vuelve a ejecutar el comando.

10. Apagar el codespace

Para qué: dejar de consumir horas de la cuota. Tus archivos, los datos y las imágenes quedan guardados.

1. Presiona `F1` (o Ctrl+Mayús+P). Se abre una caja arriba que empieza con el signo `>`: escribe a continuación `stop codespace`.
2. Elige **Codespaces: Stop Current Codespace** y presiona Enter. Abajo a la derecha aparece «Stopping codespace…».

⚠ Si la caja dice «No hay ningún resultado coincidente», le falta el signo > al principio (está buscando archivos, no comandos): escríbelo y repite.

Sugerencia: también se apaga desde https://github.com/codespaces: menú ⋯ de tu codespace → Stop codespace. Cerrar la pestaña no lo apaga: sigue encendido hasta completar 30 minutos sin actividad.

Para volver a entrar (por ejemplo, el día del taller)

1. Entra a https://github.com/codespaces: ahí aparece tu codespace con su nombre (por ejemplo, «scaling carnival»).
2. Haz clic en el nombre. Si la pestaña dice «Codespace is stopped», pulsa «Restart codespace». Reabrir tarda menos de un minuto.
3. Abre una terminal (+) y enciende solo lo que vayas a usar: `bash tpd/hadoop/encender_hadoop.sh` o `bash tpd/kafka/encender_kafka.sh`.

Sugerencia: si algo se ve raro al reabrir (una terminal que no responde), abre una terminal nueva con +: los procesos de la sesión anterior ya no existen.

Si algo falla

Lo que ves	Causa	Solución
«Extraer todo» dejó una carpeta tpd-codespace dentro de otra	Así descomprime Windows cuando ya existe una carpeta con ese nombre	Entra a la carpeta interior hasta ver los tres elementos y arrástralos desde ahí.
La portada del repositorio muestra una sola carpeta tpd-codespace	Arrastraste la carpeta en vez de su contenido	Borra el repositorio (Settings → Delete this repository) y repite el paso 2 arrastrando los tres elementos.
La portada del repositorio no muestra la carpeta .devcontainer	Faltó el paso 2.4 o el nombre del archivo quedó mal escrito	Repite el paso 2.4 con el nombre exacto .devcontainer/devcontainer.json. Si ya creaste el codespace, bórralo y créalo de nuevo.
La terminal dice «No such file or directory» con /workspaces/tpd-codespace	El repositorio tiene otro nombre	Renómbralo a tpd-codespace (Settings → Repository name → Rename), borra el codespace y créalo de nuevo.
No aparece «Create codespace on main», sino un codespace con nombre	Ya creaste uno antes	Ábrelo haciendo clic en su nombre; no crees otro.
«Building codespace…» más de 15 minutos	Conexión lenta o carga en GitHub	Recarga la pestaña (F5). Si sigue igual, vuelve a github.com/codespaces y abre el codespace desde ahí.
«Codespace is stopped»	Se apagó por inactividad (30 minutos)	Pulsa «Restart codespace».

Lo que ves	Causa	Solución
NoBrokersAvailable al usar la app, el consumidor o el simulador	Kafka no ha terminado de arrancar o quedó reiniciándose	Ejecuta bash tpd/kafka/encender_kafka.sh y espera «KAFKA LISTO».
Un job de Hadoop falla con «Cannot delete /tmp/hadoop-yarn/staging…» o «0 datanode(s) running»	Se lanzó antes de que HDFS saliera del modo seguro	Ejecuta bash tpd/hadoop/encender_hadoop.sh y repite el job cuando diga «HADOOP LISTO».
La app no abre con localhost:8501	Esa dirección es del codespace, no de tu equipo	Usa el botón «Abrir en el navegador» del aviso (paso 8.3).
[FALTA] en el verificador	Un paso quedó incompleto	La misma línea dice qué ejecutar; hazlo y repite el paso 9.
GitHub avisa que se agotó el uso incluido del mes	Se consumieron las horas gratuitas	Avisa al docente: trabajarás en pareja hasta que se renueve la cuota.

Lista de verificación final

#	Punto de control	Cumplido
1	Tienes cuenta de GitHub con el correo verificado	☐
2	Tu repositorio tpd-codespace (Private) muestra .devcontainer, tpd, README.md y devcontainer.txt	☐
3	Tu codespace está creado y la terminal muestra el indicador @tu-usuario ➜ /workspaces/tpd-codespace (main) $	☐
4	preparar_datos.py dejó el mes (19.868.009 viajes) y la semana (4.576.158) con «Punto de control correcto»	☐
5	encender_hadoop.sh terminó en «HADOOP LISTO» y /prueba/zonas.csv quedó en HDFS	☐
6	verificar_spark.py imprimió «Spark 3.5.9 funcionando»	☐
7	encender_kafka.sh terminó en «KAFKA LISTO», con «Productor OK» y «Conector Spark-Kafka OK»	☐
8	La app abrió con «Abrir en el navegador» y la central recibió el viaje	☐
9	verificar_codespace.sh terminó con «RESULTADO: LISTO para el Taller 1»	☐
10	Apagaste Kafka (docker compose down) y el codespace (Stop Current Codespace)	☐